

深圳大学实验报告

课程名称	计算机网络		
实验名称	实验 5 Socket 网络编程		
学 院	数学科学学院		
专 业	信息与计算科学（数学与计算机实验班）		
指导教师	黄耀东		
报 告 人	黄嘉聪	学号	2023192044
实验时间	2025 年 3 月 24 日-2025 年 4 月 20 日		
提交时间	2025 年 4 月 20 日		

教务处制

实验目的：

- 1.理解 TCP 套接字的定义
- 2.掌握基本的 TCP 套接字编程方法
- 3.了解简单网络应用的编程思路
- 4.了解网络编程相关的一些库

实验要求与环境：

具有 Internet 连接的主机、c 语言
了解网络编程的相关库
掌握编写简单网络应用的技能

实验内容：

- 任务一：通过套接字在 Client 和 Server 之间建立 TCP Socket 连接
任务二：在服务端使用 send() 函数向客户端发送视频文件
任务三：在客户端使用 recv() 函数接收服务端发送的视频文件

实验方法与步骤：

任务一：通过套接字在 Client 和 Server 之间建立 TCP Socket 连接

服务端：

- ①创建套接字、②配置 IP 及端口号并绑定套接字、③监听端口、④接受客户端连接请求

客户端：

- ①初始化并创建套接字
②配置 IP 及端口号
③连接服务端

任务二：在服务端使用 send() 函数向客户端发送视频文件

- ①创建发送缓冲区
②打开视频文件
③读取文件内容
④将发送缓冲区的内容发送至客户端
⑤关闭文件及 socket 连接

任务三：在客户端使用 recv() 函数接收服务端发送的视频文件

- ①创建发送缓冲区
②打开视频文件
③接收服务端发送的数据并写入接收缓冲区
④写入文件内容
⑤关闭文件及 socket 连接

各函数返回解读：

fread()、send()与 recv()、fclose()、Closesocket()

扩展任务：如何在视频流传输完成后，通知 server 结束视频传输？（client）|如何根据 client 信号，终止传输并退出循环（server）。

实验过程及内容：

任务一：通过套接字在 Client 和 Server 之间建立 TCP Socket 连接

服务端：

①创建套接字

如图一所示，Server 首先进行初始化，然后使用 Socket()函数创建一个套接字，并同时判断是否成功创建，并据此返回创建成功（失败）的提示信息。

```
// 创建套接字
if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == 0)
{
    perror("socket failed");
    exit(EXIT_FAILURE);
}
else
    printf("Create Server Socket Success.\n");
```

图 1 Server 创建套接字

②配置 IP 及端口号并绑定套接字

如图 4 所示，创建一个 sockaddr_in 类型的结构体变量来存储对应的 IP 地址信息。并如图 2 所示，将配置服务器将接受针对任意 IP 地址的请求（即设置为 INADDR_ANY 状态）以及服务器所监听的端口号（即初始定义的 PORT 号码）。

```
// 设置套接字选项
server_address.sin_family = AF_INET;
server_address.sin_addr.s_addr = INADDR_ANY;
server_address.sin_port = htons(PORT);
```

图 2 Server 设置套接字

```
#define PORT 7788
```

图 3 PORT 宏定义

```
int server_fd, new_socket;
struct sockaddr_in server_address;
int opt = 1;
int addrlen = sizeof(server_address);
char buffer[BUFFER_SIZE] = {0};
```

图 4 Server 初始化部分变量

如图 5 所示，使用 `bind()` 函数将之前创建的套接字与 `sockaddr_in` 结构体变量绑定，关联套接字和服务器的 IP 地址及端口号，并如①中套接字的创建一样，也在此处检测套接字是否绑定成功，并输出提示信息。

```
// 绑定套接字
if (bind(server_fd, (struct sockaddr *)&server_address, sizeof(server_address)) < 0)
{
    perror("bind failed");
    exit(EXIT_FAILURE);
}
else
    printf("Server Bind Port Success. \n");
```

图 5 绑定套接字

③监听端口

如图 6 所示，通过 `listen()` 函数使套接字进入监听端口图 3 中定义的 PORT，准备接收来自客户端的连接请求。

```
// 监听套接字
if (listen(server_fd, 3) < 0)
{
    perror("listen");
    exit(EXIT_FAILURE);
}
else
    printf("Server Listening.....\n");
```

图 6 监听客户端请求

④接受客户端连接请求

如图 7 所示，当服务端监听到端口上的连接请求时，使用 `accept()` 函数接收连接并返回一个新的连接 SOCKET(套接字描述符)。这个新 SOCKET 用于服务器和客户端之间的后续通信。

```
if ((new_socket = accept(server_fd, (struct sockaddr *)&server_address, &addrlen)) < 0)
{
    perror("accept");
    exit(EXIT_FAILURE);
}
else
    printf("Server Accept Success. \n");
```

图 7 服务器端判断接受请求并返回新的 SOCKET

客户端：

①初始化并创建套接字

如图 8 所示，与服务端相同，使用 `socket()` 函数创建一个套接字，并返回套接字描述符。

```
if ((sock = socket(AF_INET, SOCK_STREAM, 0)) < 0)
{
    perror("Socket creation error");
    return -1;
}
else
    printf("Client Create Socket Success. \n");
```

图 8 Client 创建套接字

②配置 IP 及端口号

如图 x，与服务端相同，创建一个 `sockaddr_in` 类型的结构体变量来存储地址信息。并配置请求的服务器 IP 地址及端口号。

```
serv_addr.sin_family = AF_INET;  
serv_addr.sin_port = htons(PORT);  
serv_addr.sin_addr.S_un.S_addr = inet_addr("127.0.0.1");
```

图 9 Client 设置地址

③连接服务端

如图 x 所示，客户端使用 `connect()` 函数向服务端发送连接请求。

```
if (connect(sock, (struct sockaddr *)&serv_addr, sizeof(serv_addr))  
{  
    perror("Connection Failed");  
    return -1;  
}  
else  
    printf("Client Connect Server Success. \n");
```

图 10 Client 发送连接请求

任务二：在服务端使用 `send()` 函数向客户端发送视频文件

①创建发送缓冲区

如图 4 所示，创建一个 `char` 类型的数组作为发送缓冲区，存储待发送的数据。

`char buffer[BUFFER_SIZE] = {0};`

②打开视频文件

如图 12，使用 `FILE *fopen (const char *filename, const char *mode);` 打开视频文件。其中 `filename` 表示代传输的视频文件名，在本次实验中使用图 11 所示的宏定义变量进行输入。

```
// 找到文件  
char *file_name = req;  
char file_path[100] = VIDEO_PATH;  
strcat(file_path, file_name);
```

图 11 Server 确定文件完全地址

```
// 打开视频文件  
FILE *fp = fopen(file_path, "rb"); // 以二进制读模式打开文件  
if (fp == NULL)  
{  
    perror("File open failed");  
    continue;  
}
```

图 12 Server 打开视频文件

③读取文件内容

如图 13，使用 `size_t fread (void * ptr, size_t size, size_t count, FILE * stream);` 读取视频文件内容，并放入发送缓冲区。该函数返回成功读取的元素总数。如果此数字与计数参数不同，则可能发生读取错误或读取时已到达文件结尾，此时对该种情况进行提示并结束程序。

```
// 从文件读取数据到缓冲区
int bytes_read = fread(buffer, 1, bytes_to_send, fp);
if (bytes_read <= 0) {
    printf("文件读取错误或已到达文件尾\n");
    break;
}
```

图 13 Server 读取文件内容至缓冲区

④将发送缓冲区的内容发送至客户端

如图 14，使用 `int WSAAPI send(SOCKET s, const char *buf, int len, int flags);` 将发送缓冲区的内容发送至客户端。该函数返回的内容为传输的字符个数，此时如图 15，此时可利用该返回数，进行传输错误判断与进度条的显示。其中，对于进度条使用 `\r` 对输出内容进行重置，达到类似进度条的效果。

```
// 发送缓冲区数据
bytes_sent = send(new_socket, buffer, bytes_read, 0);
```

图 14 Server 发送缓冲区数据至客户端

```
if (bytes_sent < 0) {
    printf("ERROR in send data, 错误码: %d\n", WSAGetLastError());
    break;
}
send_count += bytes_sent;
// 显示发送进度
printf("\r发送进度: %.2f%%", (float)send_count / file_size * 100);
fflush(stdout);
```

图 15 Server 中 send 函数返回值的使用

⑤关闭文件及 socket 连接

如图 16 所示，在传输完文件结束符后，使用 `int fclose(FILE * stream);` 关闭文件指针。如果流成功关闭，则返回 0。失败时，则返回系统宏定义 EOF。

```
// 发送文件结束符
unsigned char s_stop_byte = STOP_BYTE;
bytes_sent = send(new_socket, &s_stop_byte, sizeof(s_stop_byte), 0);
if (bytes_sent < 0)
    printf("ERROR in send stop byte, 错误码: %d\n", WSAGetLastError());

// 关闭文件
fclose(fp);
printf("文件已关闭\n");
```

图 16 Server 关闭文件

如图 17 所示，使用 `int WSAAPI closesocket(SOCKET s);` 关闭 socket 连接。s 表示要关闭的套接字的描述符。如果没有发生错误，`closesocket()` 返回 0； 否则返回系统宏定义 `SOCKET_ERROR`。

```
/**结束阶段**/  
closesocket(server_fd);  
closesocket(new_socket);  
WSACleanup();
```

图 17 Server 关闭 socket

任务三：在客户端使用 `recv()` 函数接收服务端发送的视频文件

①创建发送缓冲区

如图 9 所示，类似于服务端，创建一个 `char` 类型的数组作为发送缓冲区，存储待发送的数据。即 `char buffer[BUFFER_SIZE] = {0};`

②打开视频文件

使用 `FILE *fopen (const char * filename, const char * mode );` 创建一个空的文件，用于存储接收到的视频数据。

```
FILE *fp = fopen(file_path, "wb"); // 以二进制模式打开文件，并返回文件指针  
if (fp == NULL)  
{  
    perror("fopen");  
    exit(EXIT_FAILURE);  
}
```

图 18 Client 创建文件

③接收服务端发送的数据并写入接收缓冲区

使用 `int WSAAPI recv(SOCKET s, const char *buf, int len, int flags);` 接收到的数据读入接收缓冲区，然后将其复制到视频片段缓冲区内以便写入。

```
// 接收数据到缓冲区  
bytes_recv = recv(sock, buffer, bytes_to_receive, 0);  
if (bytes_recv <= 0) {  
    printf("接收数据错误或连接关闭\n");  
    break;  
}  
// 将接收到的数据复制到视频片段缓冲区中  
memcpy(video_segement + recv_count, buffer, bytes_recv);
```

图 19 Client 读取文件并写入缓冲区内

如果没有发生错误，`recv()` 返回接收到的字节数，buf 参数指向的缓冲区将包含接收到的数据。同服务端，如图 19 所示，利用 `recv()` 接受到的字节数，制作进度条。

```
recv_count += bytes_recv;  
// 显示下载进度  
printf("\r接收进度: %.2f%%", (float)recv_count / file_size * 100);  
fflush(stdout);
```

图 20 Client 进度条

④写入文件内容

如图 21，使用 `size_t fwrite (const void * ptr, size_t size, size_t count, FILE * stream);` 将视频片段缓冲区内的数据写入创建的视频文件中。

```
fwrite(video_segement, 1, file_size, fp);
```

图 21 Client 将缓冲区写入文件中

⑤关闭文件及 socket 连接

如图 22 所示，释放视频缓冲流；如图 23，使用 `int fclose(FILE * stream)` 关闭文件指针；如果图 24 所示，关闭 socket 连接。

```
free(video_segement);  
video_segement = NULL;
```

图 22 Client 释放视频缓冲流

```
fclose(fp); // 关闭文件
```

图 23 Client 关闭文件指针

```
closesocket(sock);  
WSACleanup();
```

图 24 Client 关闭 socket 连接

各函数返回解读：

`fread()`: 该函数返回成功读取的元素总数。如果此数字与计数参数不同，则可能发生读取错误或读取时已到达文件结尾，据此，可使用其与记录的参数进行比较，如果不同则认为发生了读取错误，或者达到文件结尾，也即需停止读取，同时也可进行调试信息的输出。

`send()`与 `recv()`: 该函数返回发送/接受到的字节数，如任务二、三中所示，其与实际发送的字节数据进行比较，不仅可以作为函数运行正常的判断，也可用作进度条的使用

`fclose()`: 如果流成功关闭，则返回 0。失败时，返回系统宏定义 `EOF`。`EOF` 是系统固定的宏定义，表示文件结尾，这样尽管流未成功关闭，但是在文件的读取上可以使得流的状态不影响文件的后续读取。当然，也可以直接用于是否关闭的判断。

`Closesocket()`: 如果没有发生错误，返回 0； 否则返回系统宏定义 `SOCKET_ERROR`。不同的返回可以作为 `socket` 是否正常关闭的判断，同时固定的系统宏定义也可以对后面调试信息的输出，如获取到 `SOCKET_ERROR` 就可知是 `socket` 出了问题，便可进行相对应的排查。

扩展任务：如何在视频流传输完成后，通知 `server` 结束视频传输？（`client`）
| 如何根据 `client` 信号，终止传输并退出循环（`server`）。

如图 25、26 所示，即在传输完毕后，客户端主动向服务器发送固定的一个代码，服务器收到后便直接停止传输并结束程序。为了服务器可以随时判断，该接受判断位于接受主循环中。

```
char end_request[REQUEST_SIZE] = "END_REQUEST";  
bytes_sent = send(sock, end_request, REQUEST_SIZE, 0);
```

图 25 Client 发送结束视频传输代码


```

if (strcmp(req, END_REQUEST) == 0) {
    printf("收到客户端终止请求，服务器将停止传输\n");
    break; // 退出主循环
}

```

图 25 Server 接受结束视频传输代码

实验结果：

如图 26、27 所示，编译并同时运行文件后，视频被正确传输过去。

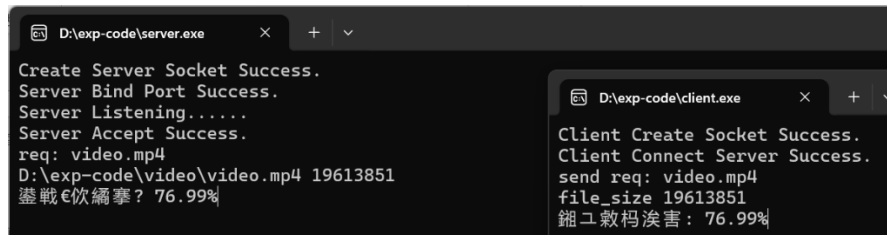


图 26 程序运行

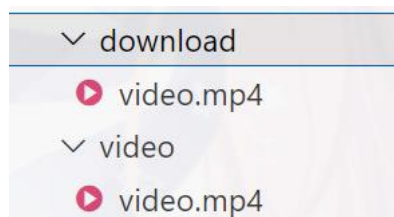


图 27 程序运行结果

深圳大学学生实验报告用纸

实验结论：

- ①通过本次实验，成功完成了基于 TCP 套接字的网络编程任务，实现了客户端与服务端之间的稳定连接。服务端通过 **send()** 函数分块发送视频文件，客户端通过 **recv()** 函数接收并写入本地，验证了 TCP 协议可靠传输的特性。
 - ②文件读写过程中结合缓冲区机制优化了传输效率，并通过 **fread()** 和 **fwrite()** 的返回值判断传输状态（如文件结束或错误）。在扩展任务中，客户端通过发送自定义结束信号通知服务端终止传输，服务端根据信号退出循环并关闭连接，实现了传输结束的主动通知。
 - ③在传输完整性上，通过比对 **send()/recv()** 返回值与预期字节数确保数据无丢失，利用 **feof()** 检测文件结束标志；在资源管理上使用 **fclose()** 和 **closesocket()** 规范释放资源，避免内存泄漏或端口占用；在调试优化上基于返回值动态刷新进度条，直观展示传输进度。
 - ④实验收获包括掌握 TCP 套接字编程的核心函数（**socket()**、**bind()**、**listen()**、**accept()**、**connect()** 等），理解客户端与服务端的协作逻辑。
 - ⑤改进方向：引入多线程技术实现服务端并发处理多个客户端请求，增加类似于 HTTP 中的异常处理机制（如超时重传、断点续传）提升传输可靠性。
- 综上，本实验通过实践验证了 TCP 套接字编程方法，完成了本次实验。

指导教师批阅意见:

成绩评定：

指导教师签字:

年 月 日

备注:

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。